

现代编程思想

命令式编程

月兔公开课课程组

函数式编程

- 到此为止，我们介绍的编程知识可以归类于函数式编程的范畴
 - 对每一个输入，有着固定的输出
 - 对于标识符，我们可以直接用它所对应的值进行替代——引用透明性
- 开发实用的程序，我们需要一些计算之外的“副作用”
 - 进行输入输出
 - 修改内存中的数据等
 - 这些副作用可能导致多次执行的结果不一致

引用透明性

- 我们可以定义如下数据绑定和函数

```
1. let x : Int = 1 + 1
2. fn square(x : Int) -> Int { x * x }
3. let z : Int = square(x) // 4
```

- 我们可以将 `square` 与 `x` 直接用对应的值替换而不改变结果

```
1. let z : Int = { 2 * 2 } // 4
```

- 引用透明性可以易于理解

- 函数 `println` 允许我们输出一个字符串，例如 `println("hello moonbit")`
- 月兔中 `main` 函数即为程序入口

```
1. fn main {  
2.     println("hello moonbit")  
3. }
```

Debug

```
fn main {  
    println("hello moonbit")  
}
```

hello moonbit

命令与副作用

- 输出命令可能会破坏引用透明性

```
fn square(x : Int) -> Int { x * x }
```

🐛 Debug

```
fn main {  
  let x : Int = { x = 2  
    println("hello moonbit") // <-- 我们首先执行命令，进行输出  
    1 + 1 // <-- 之后，我们以表达式块最后的值作为表达式块的值  
  }  
  let z : Int = square(x) // 4, 输出一次 z = 4  
}
```

```
hello moonbit
```

命令与副作用

- 我们不一定可以放心替换，因此会增大程序理解难度

Debug

```
fn main {  
  let z : Int = { z = 4  
    println("hello moonbit") // <-- 进行第一次输出  
    1 + 1 // <-- 获得值: 2  
  } * {  
    println("hello moonbit") // <-- 进行第二次输出  
    1 + 1 // <-- 获得值: 2  
  } // 4, 输出了两次  
}
```

```
hello moonbit  
hello moonbit
```

单值类型

- 我们之前已经介绍过单值类型 `Unit`
 - 它仅有一个值: `()`
- 以 `Unit` 为运算结果类型的函数或命令一般有副作用
 - `fn println(String) -> Unit`
- 命令的类型也是单值类型

```
1. fn do_nothing() -> Unit {  
2.   let _x = 0 // 结果为单值类型, 符合函数定义  
3. }
```

变量

- 在月兔中，我们可以在代码块中用 `let mut` 定义临时变量

```
1. test {  
2.   let mut x = 1  
3.   x = 10 // 赋值操作是一个命令  
4. }
```

- 在月兔中，结构体的字段默认不可变，我们也允许可变的字段，需要用 `mut` 标识

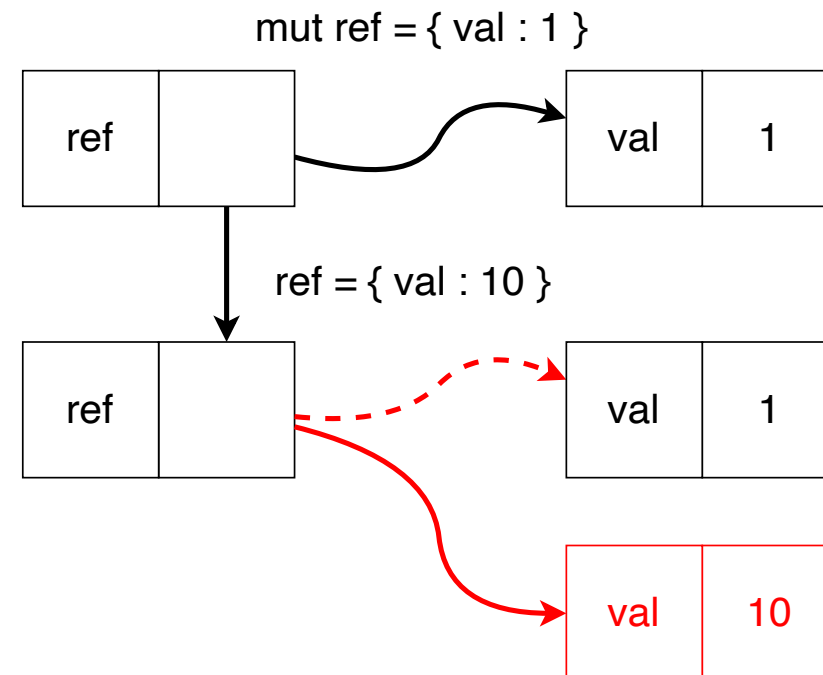
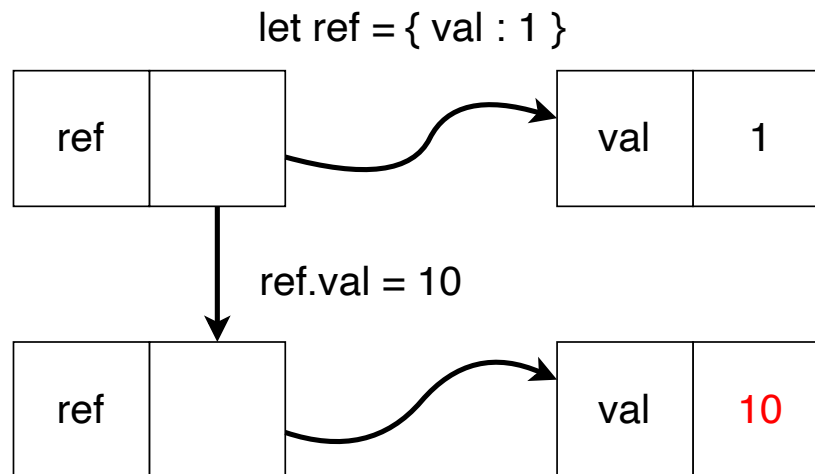
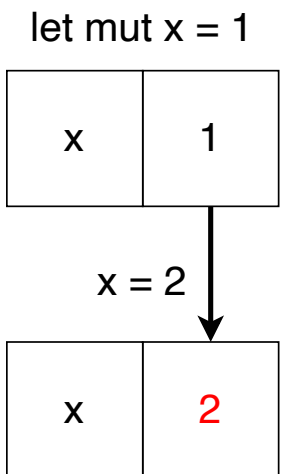
```
1. struct Ref[T] { mut val : T }  
2.  
3. test {  
4.   let ref : Ref[Int] = { val : 1 } // ref 本身只是一个数据绑定  
5.   ref.val = 10 // 我们可以修改结构体的字段  
6.   inspect(ref.val, content="10") // 输出 10  
7. }
```


- 枚举类型带标签的负载值也可以标识为 `mut`

```
1. enum List[T] {  
2.   Nil  
3.   Cons(mut head~ : T, mut tail~ : List[T])  
4. } derive(Show)  
5.  
6. test {  
7.   let a = Cons(head=1, tail=Nil)  
8.   let b = Cons(head=2, tail=Nil)  
9.   match b {  
10.    Nil => panic()  
11.    Cons(_) as cons => {  
12.      cons.head = 2  
13.      cons.tail = a  
14.    }  
15.  }  
16.  inspect(b, content="Cons(head=2, tail=Cons(head=1, tail=Nil))")  
17. }
```

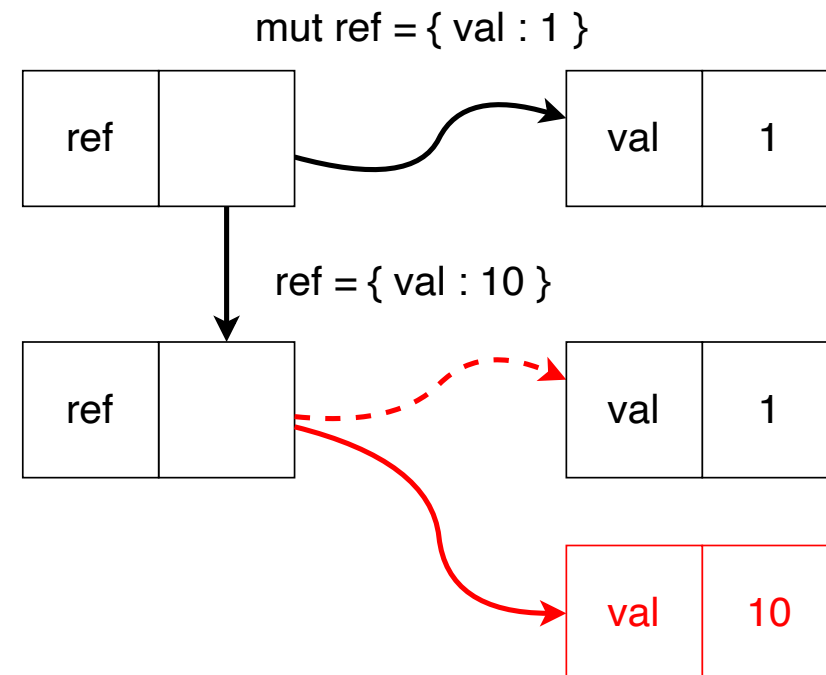
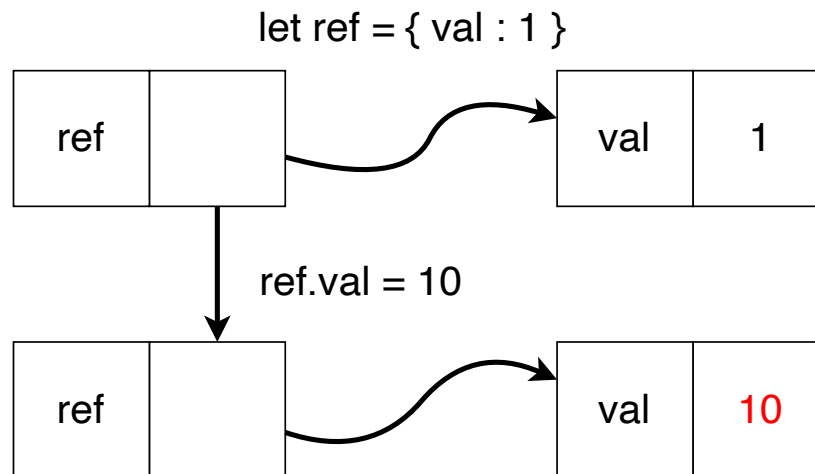
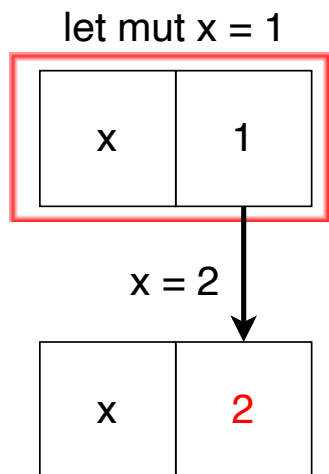
变量

- 我们可以将带有可变字段的结构体看作是引用



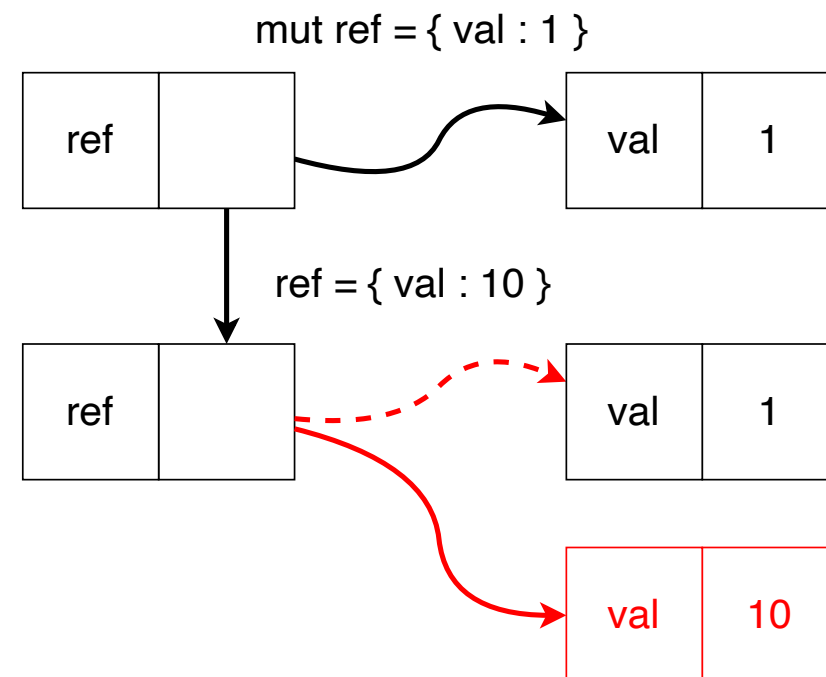
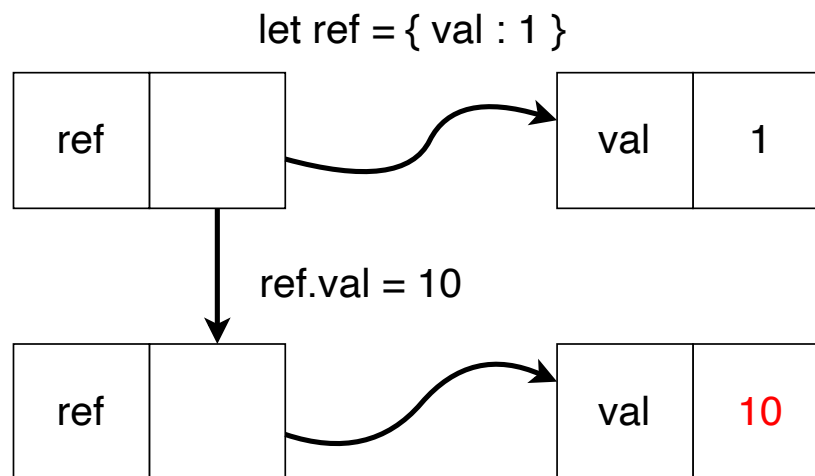
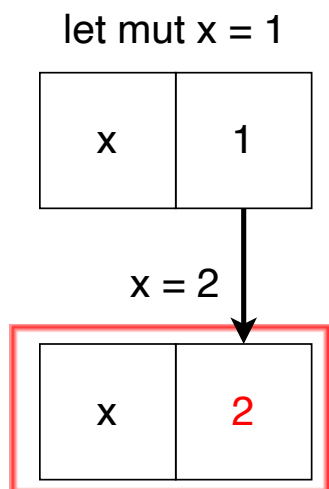
变量

- 我们可以将带有可变字段的结构体看作是引用



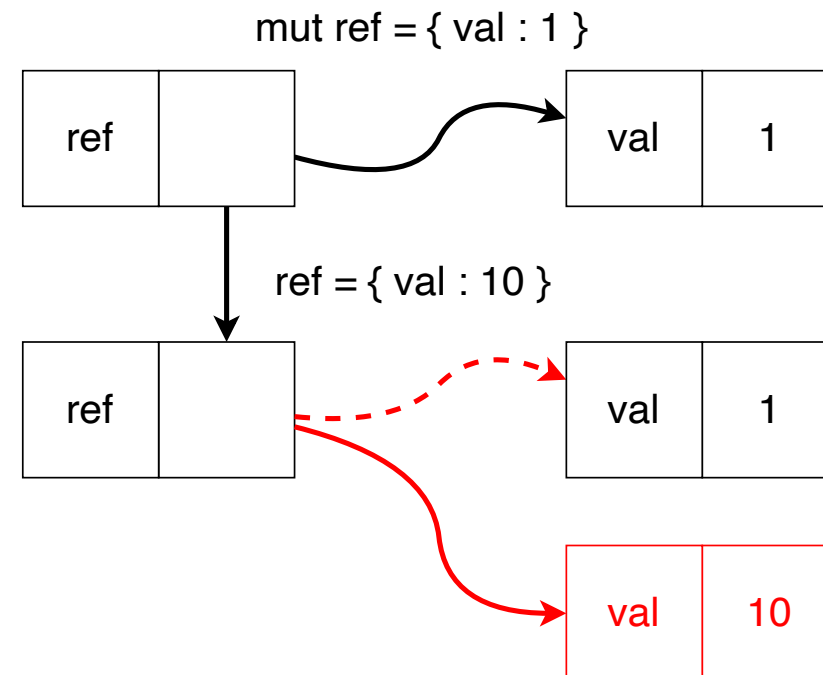
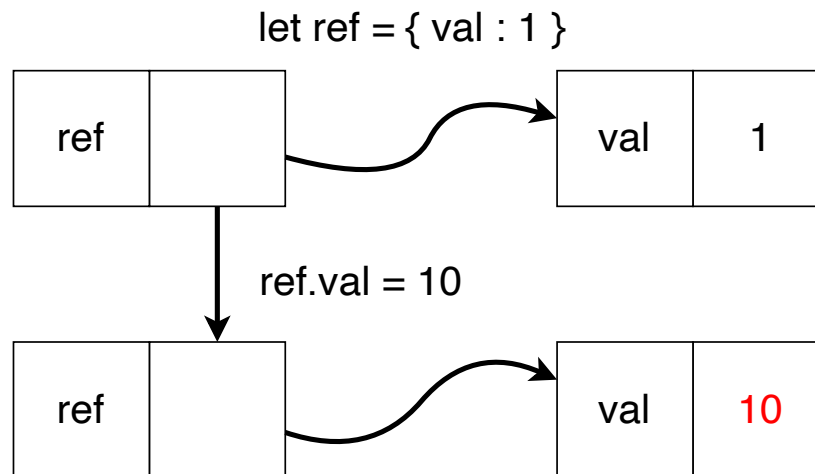
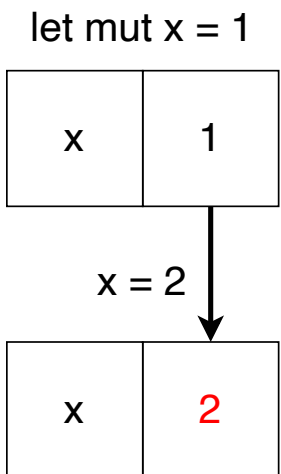
变量

- 我们可以将带有可变字段的结构体看作是引用



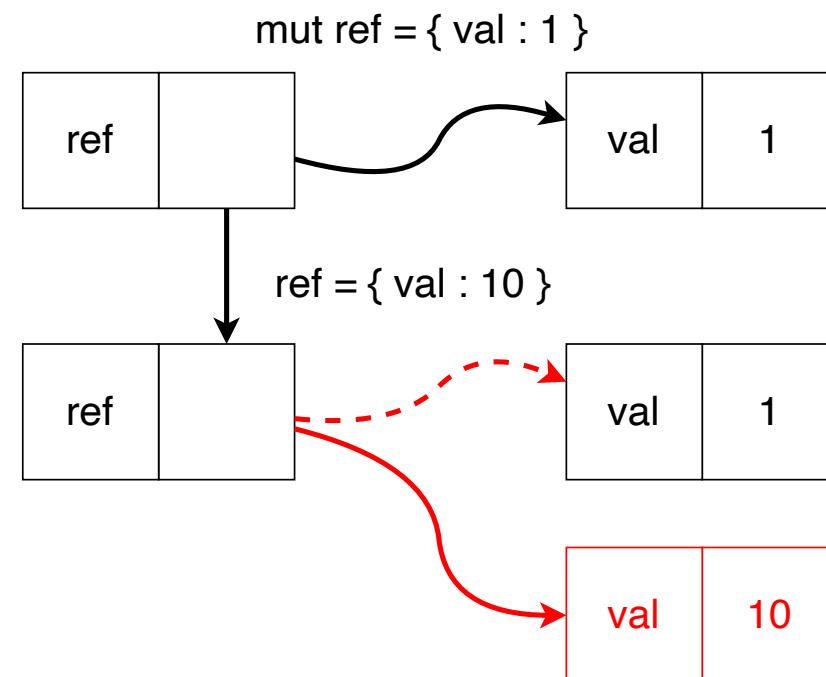
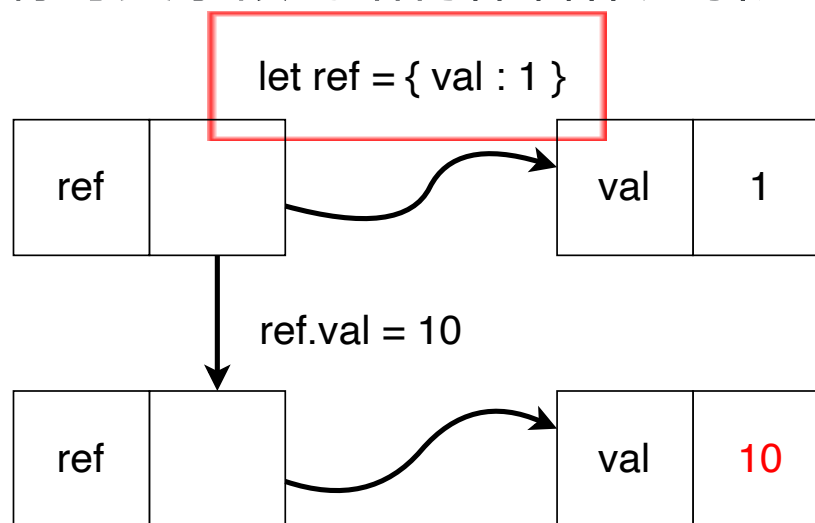
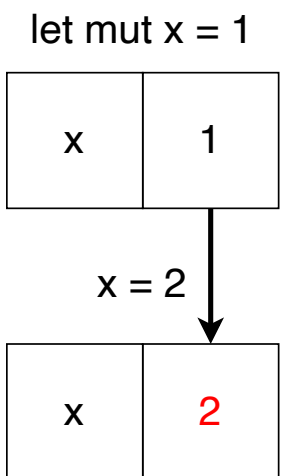
变量

- 我们可以将带有可变字段的结构体看作是引用



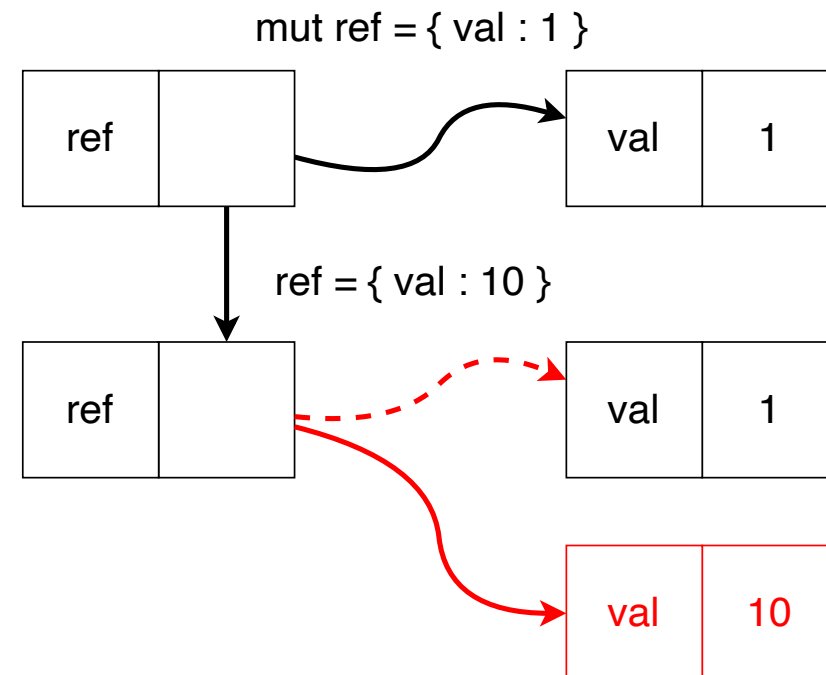
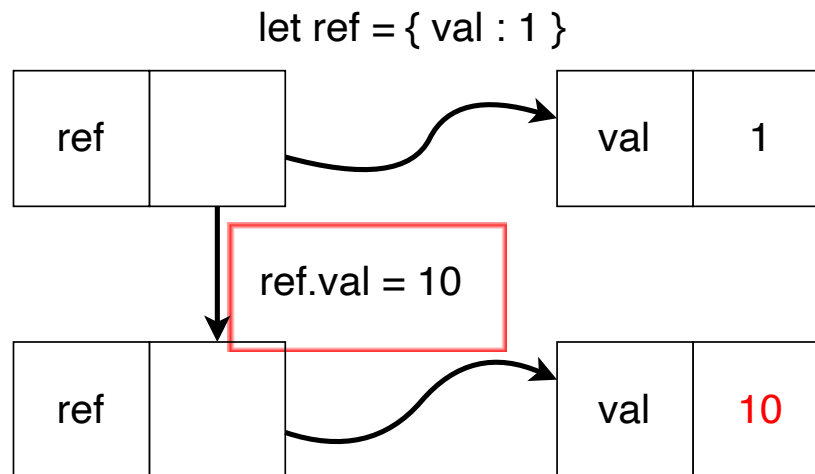
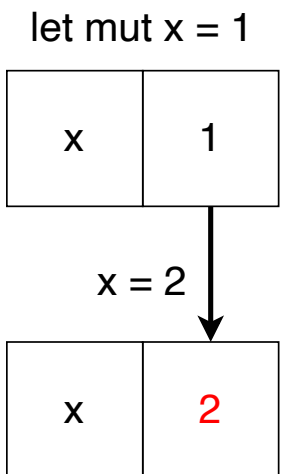
变量

- 我们可以将带有可变字段的结构体看作是引用



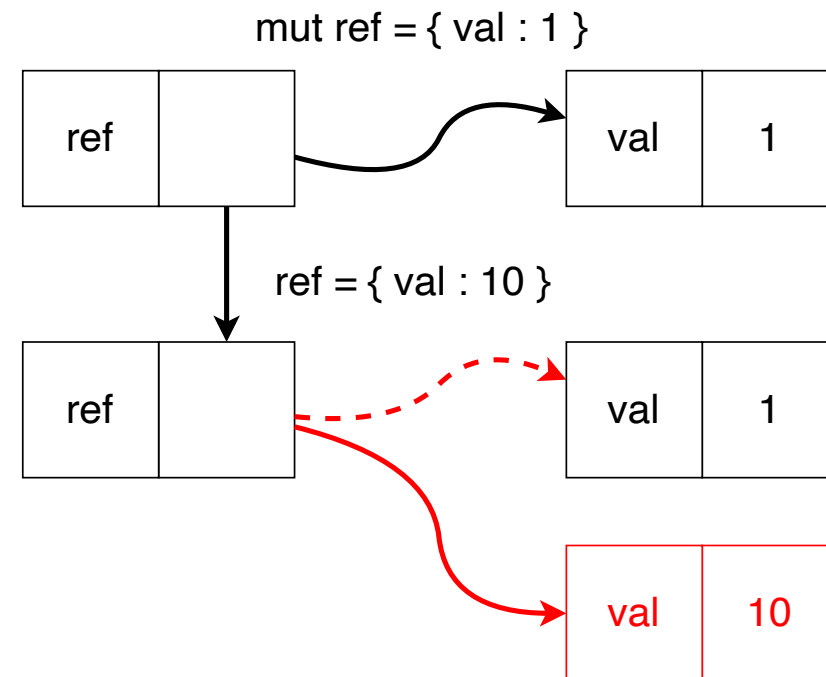
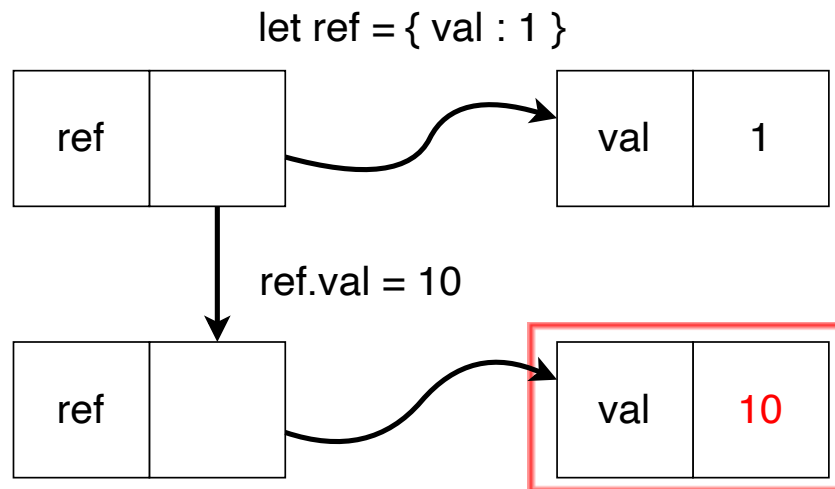
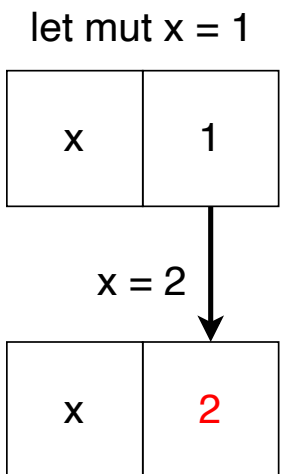
变量

- 我们可以将带有可变字段的结构体看作是引用



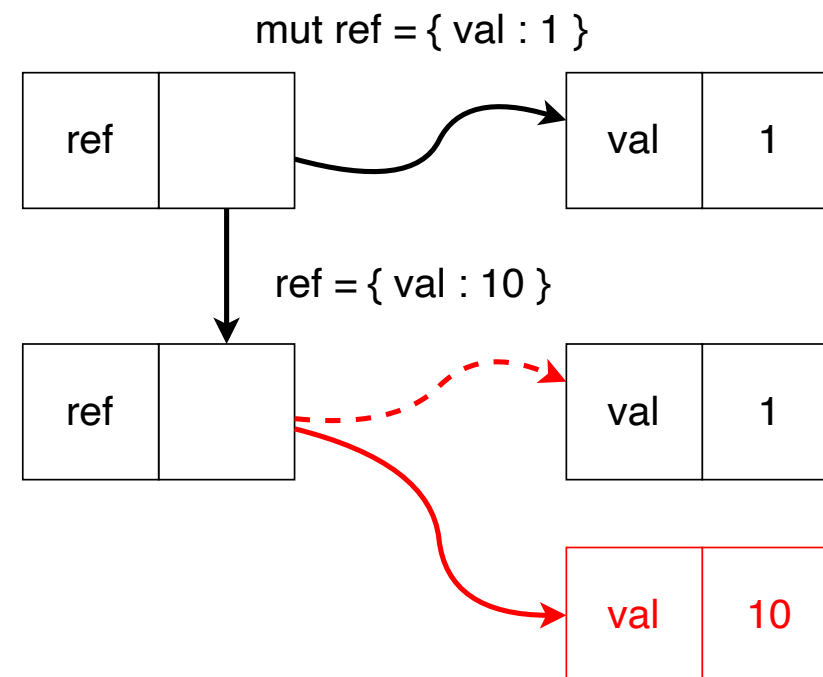
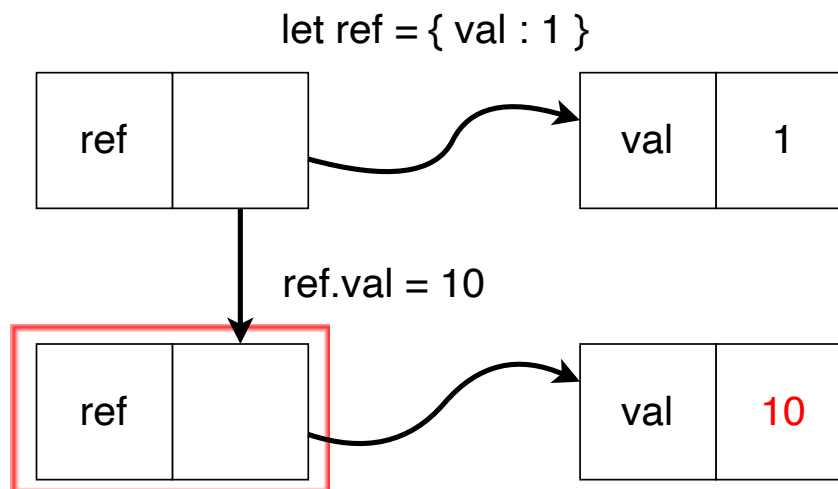
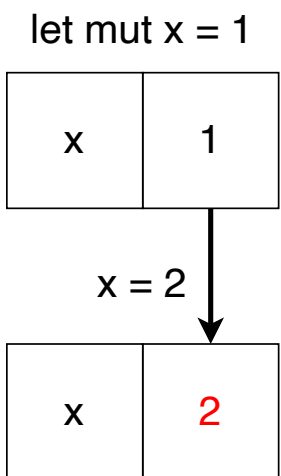
变量

- 我们可以将带有可变字段的结构体看作是引用



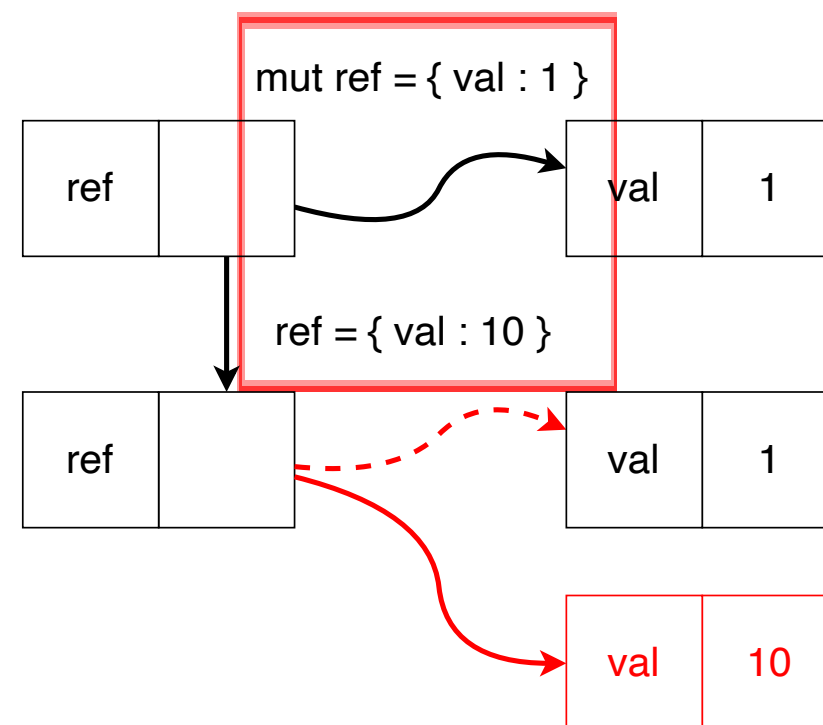
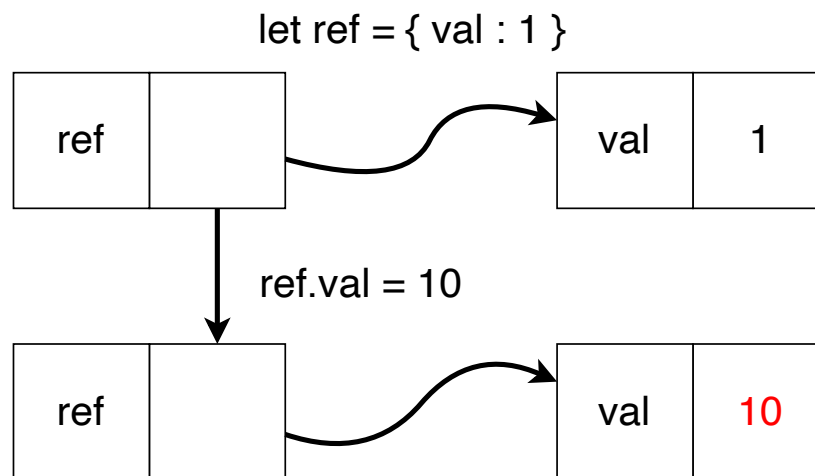
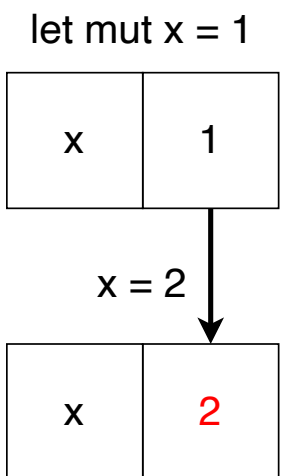
变量

- 我们可以将带有可变字段的结构体看作是引用



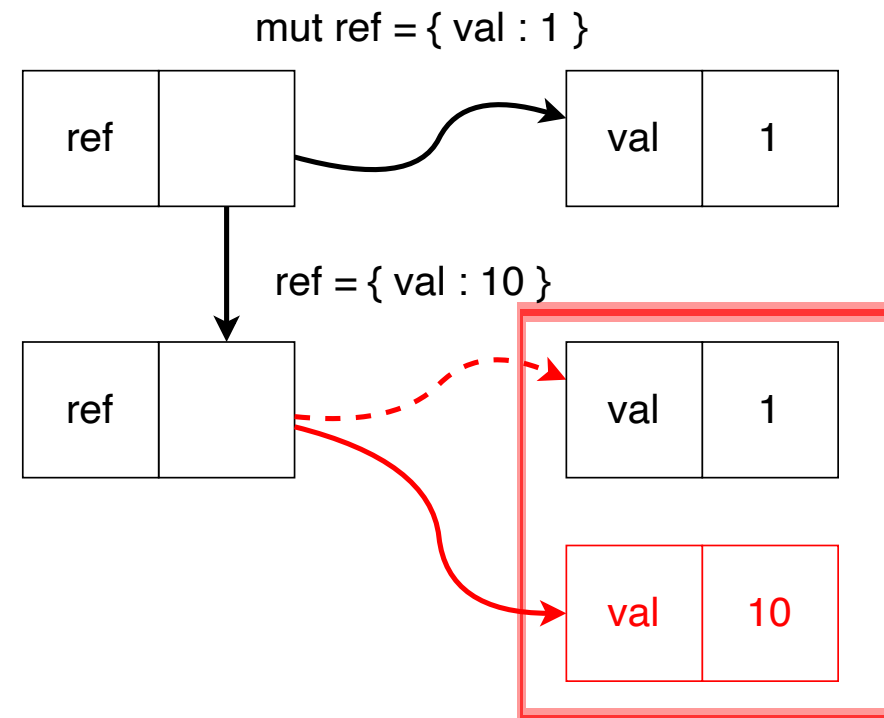
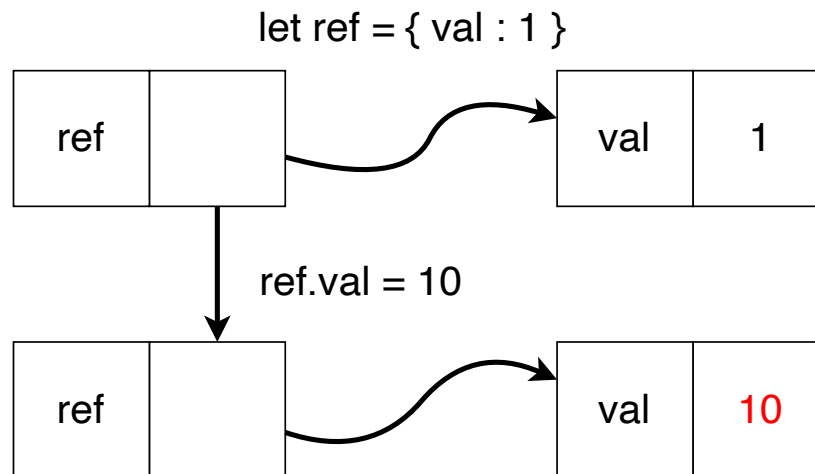
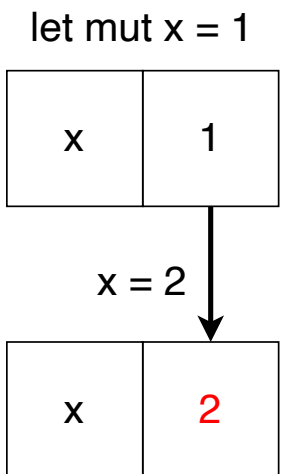
变量

- 我们可以将带有可变字段的结构体看作是引用



变量

- 我们可以将带有可变字段的结构体看作是引用



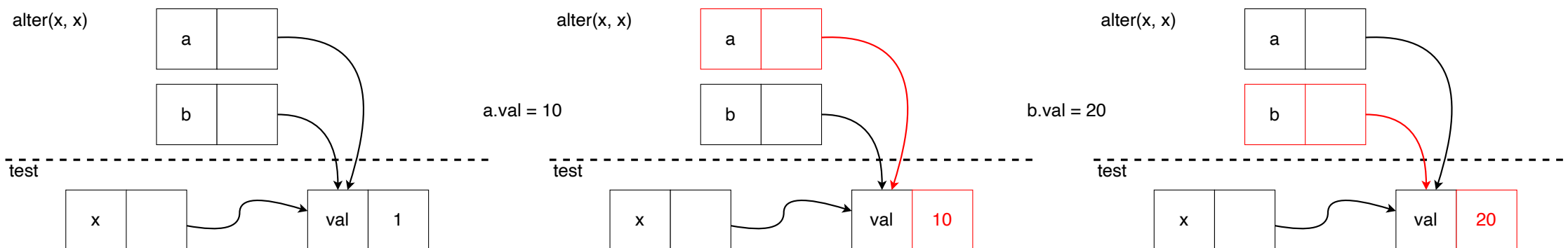
别名

- 指向相同的可变数据结构的两个标识符可以看作是别名

```
1. fn alter(a : Ref[Int], b : Ref[Int]) -> Unit {  
2.   a.val = 10  
3.   b.val = 20  
4. }  
5.  
6. test {  
7.   let x : Ref[Int] = { val : 1 }  
8.   alter(x, x)  
9.   inspect(x.val, content="20") // x.val的值将会被改变两次  
10. }
```

别名

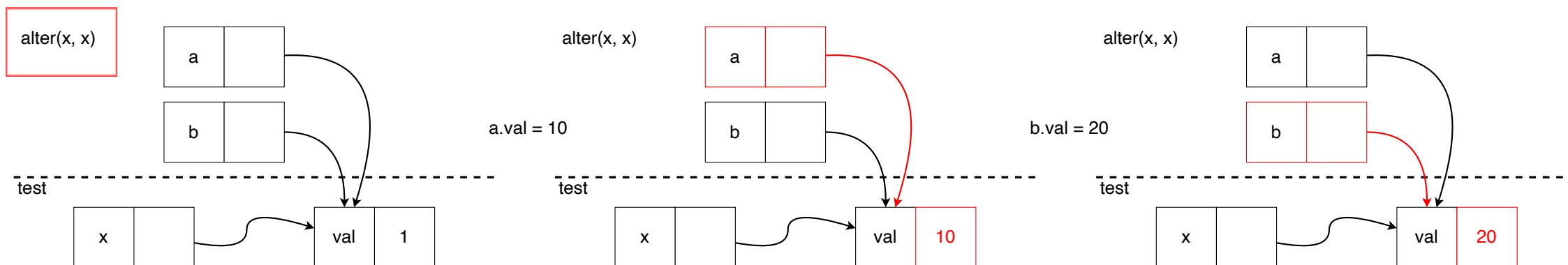
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

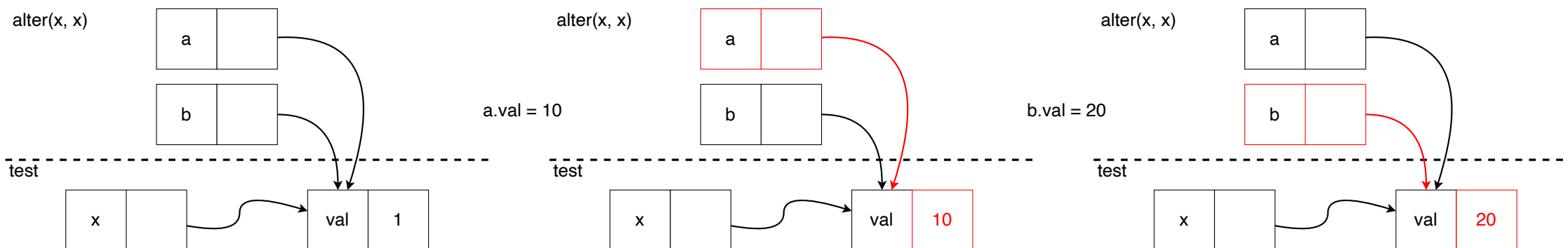
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

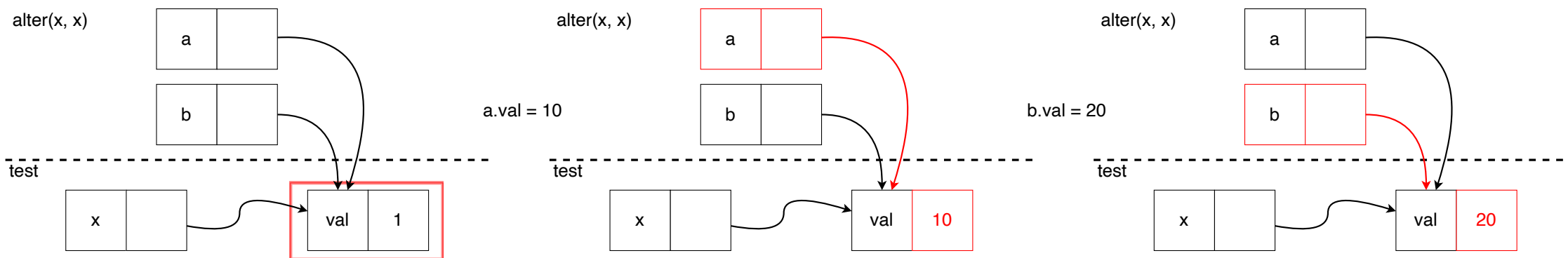
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

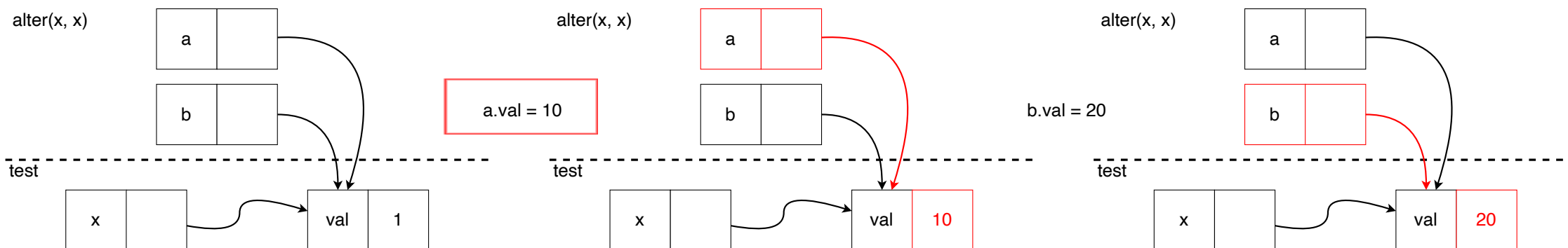
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

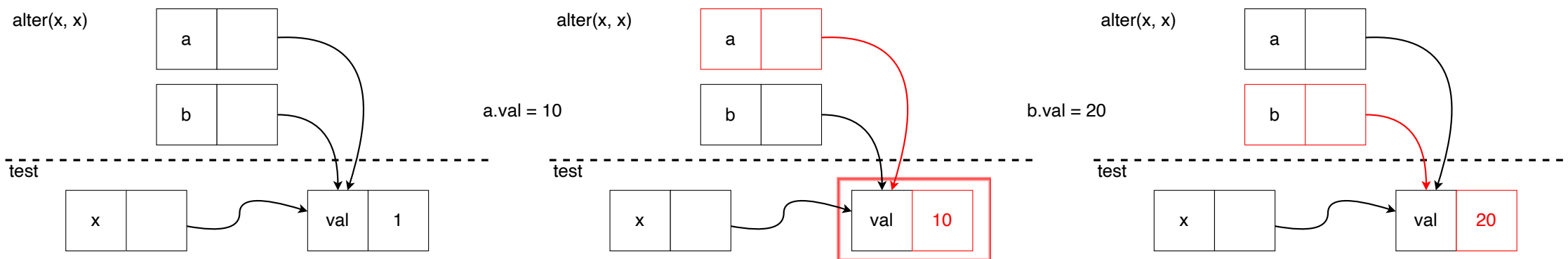
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

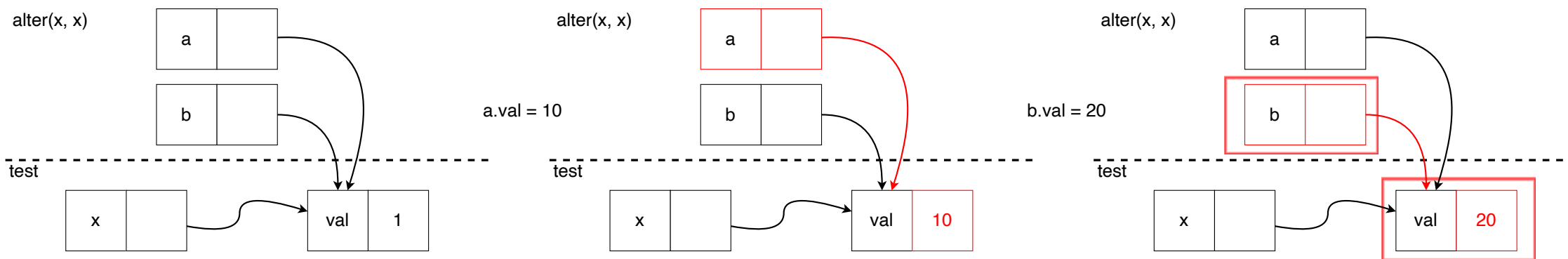
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

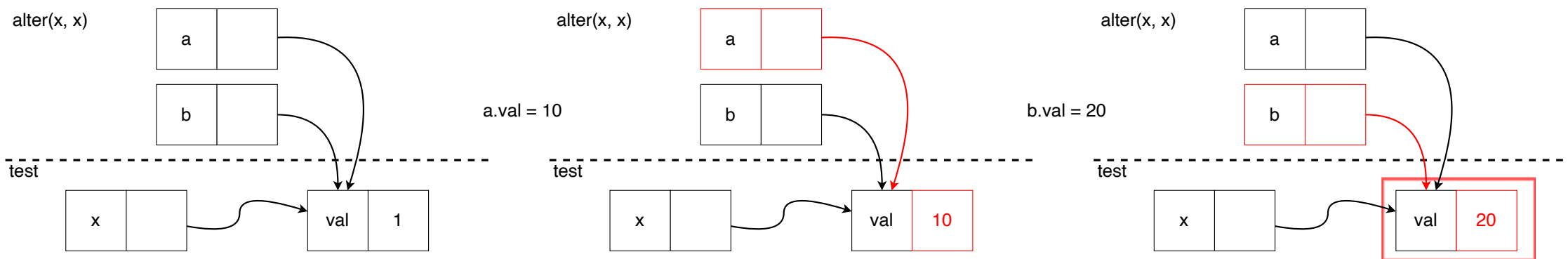
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

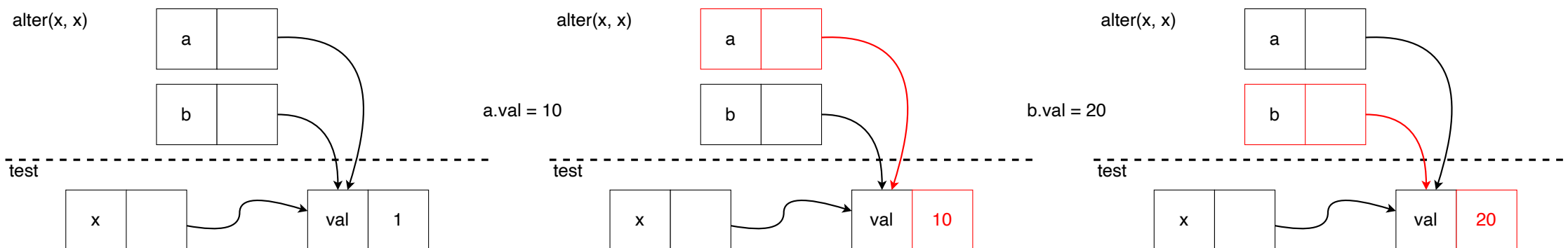
- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理

别名

- 指向相同的可变数据结构的两个标识符可以看作是别名



- 可变变量需要小心处理